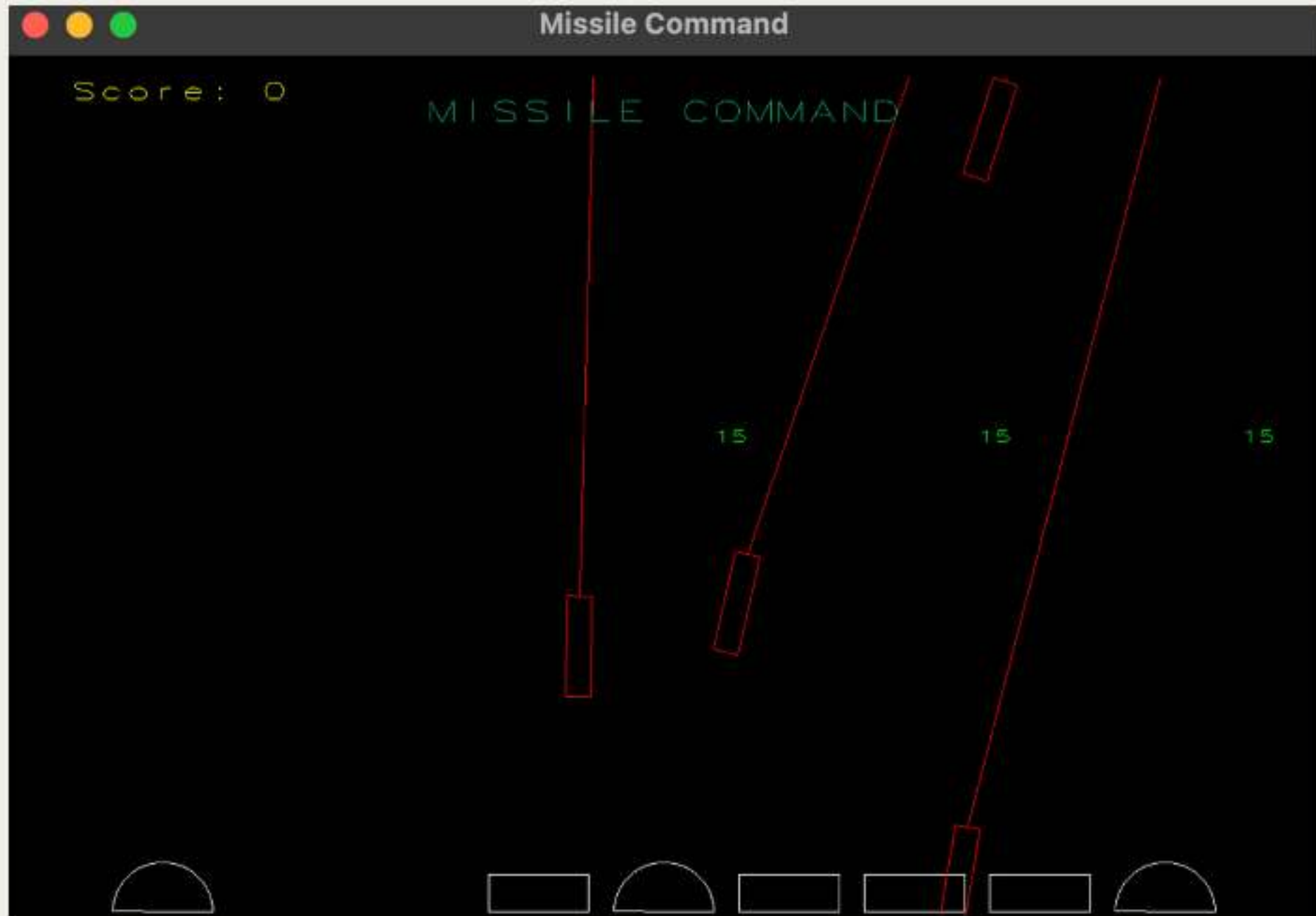


Missile Command



Missile Command is a classic Atari game, It was very impressive in my childhood.

I re-implemented it based on C++ and OpenGL, and add supports to multi-platforms.

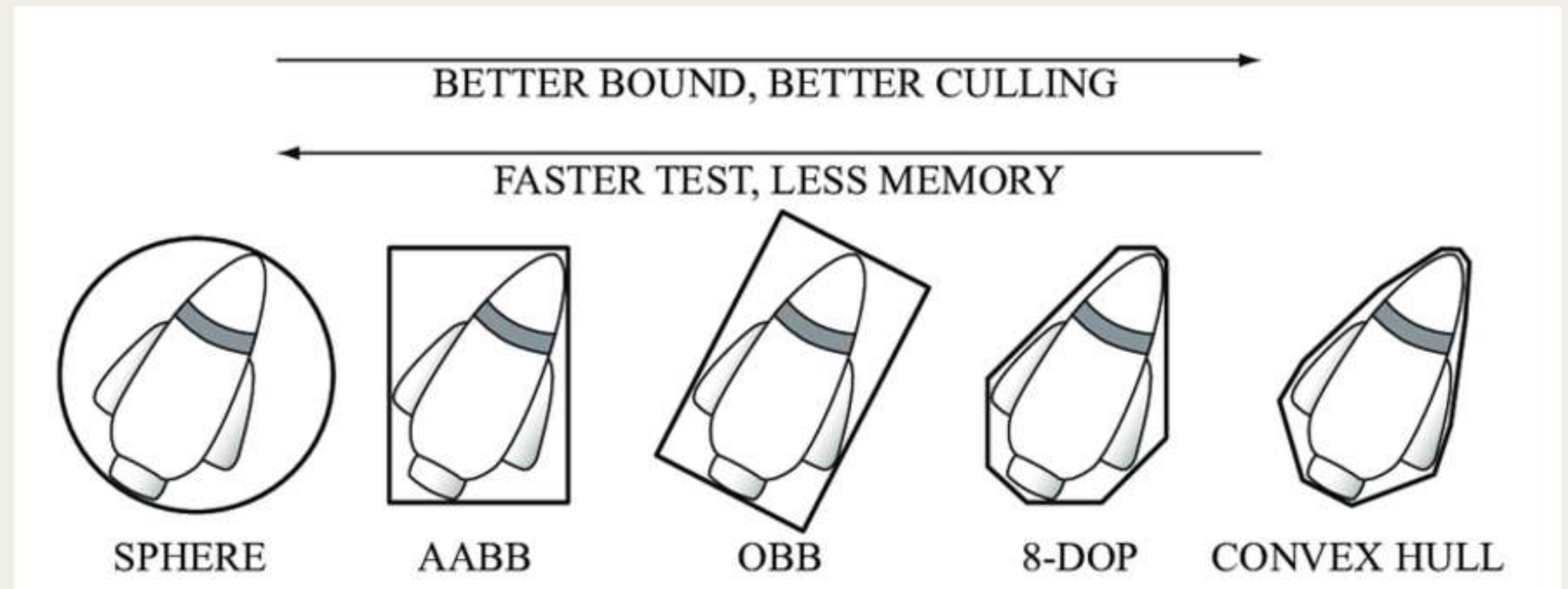
Gameplay

In the game, players need to control three turrets (white semicircles) to destroy enemy missiles (red) and protect our city (white squares).

What is different from the original is that players need to use the D/F/G buttons to select the controlled turret, and click the mouse on the target direction of our missiles.

Tech

I use the AABB algorithm to calculate collisions between missiles and cities.



Code

```
void World::handleKeystroke(int key, float x, float y) {  
  
    vec4 screenPos(x, y, 0, 1);  
    vec4 worldPos = worldToScreen.inverse() * screenPos;  
    vec3 mouseWorld(worldPos.x, worldPos.y, 0);  
  
    switch (key) {  
    case 'S': fireMissile(0, mouseWorld); break;  
    case 'D': fireMissile(1, mouseWorld); break;  
    case 'F': fireMissile(2, mouseWorld); break;  
    }  
}
```

I capture the mouse input position and convert it to world space, then fire the missile.

The fireMissile function is more like a “command”, it generates the new missile and pushes it into the queue.

The world update, will update all items state, calculate collision, remove destroyed items and render everything.

```
void World::fireMissile(int siloIndex, vec3 worldMousePos)  
{  
  
    if (!silos[siloIndex].isAlive())  
        return;  
    if (!silos[siloIndex].canShoot())  
        return;  
  
    vec3 siloPos = silos[siloIndex].position();  
    vec3 toMouse = worldMousePos - siloPos;  
    if (toMouse.length() < 1e-4f)  
        return;  
  
    vec3 dir = toMouse.normalize();  
    vec3 vel = INIT_OUTGOING_MISSILE_SPEED * dir;  
  
    Missile outMissile(siloPos, vel, -9999.f, vec3(0,1,0));  
    missilesOut.add(outMissile);  
  
    silos[siloIndex].decrMissiles();  
}
```